ПРАКТИЧЕСКОЕ ЗАДАНИЕ · ХАКАТОН

Система анализа изменений

Сервис / прототип | Срок: 2 дня | Команда: 3–5 человек | Стек: ваш выбор

ПРЕДМЕТНАЯ ОБЛАСТЬ

Что такое СПР — на простом примере

Иван оформляет микрозайм в приложении на телефоне. Пока он смотрит на спиннер, за две секунды **система принятия решений** успевает: запросить его кредитную историю в бюро, посчитать по его данным балл — вероятность, что он вернёт деньги, — прогнать заявку по списку правил-отсекателей, сравнить балл с порогом и подобрать продукты тех партнёров, которым такой клиент подходит. На выходе — решение: одобрить, отказать или отправить на проверку. И если одобрить, то какой продукт и на какую сумму.

Набор этих правил, моделей и порогов называется **стратегией**. Вот как она сработала на двух заявках:

Иван, 19 лет ОТКАЗ

→ правило «клиенту меньше 21 года» — сработало
→ дальше заявка не считается

Мария, 34 года ОДОБРЕНО

→ правила не сработали
→ балл 0.69 при пороге 0.55 — прошла
→ лимит 100 000

Стратегию правят почти каждую неделю: убирают одно правило, добавляют другое, перекатывают модель, двигают порог. И каждая такая правка меняет решения по живым людям. Убрали правило про 21 год — часть тех, кому раньше отказывали, теперь получит одобрение. Подняли порог — наоборот, часть одобренных уйдёт в отказ. **Заранее не видно, кого и как это затронет** — а узнавать об этом после выката на прод слишком дорого.

Поэтому перед выкатом берут несколько тысяч исторических заявок и прогоняют их через обе версии стратегии. Проблема в том, что ответ СПР по одной заявке — это документ на сотню с лишним атрибутов, и сравнить несколько тысяч пар таких документов глазами невозможно.

КАК ЭТО ВЫГЛЯДИТ ТЕХНИЧЕСКИ

СПР — это сервис с API: на вход JSON-документ с данными заявки, на выход JSON-документ с решением и всем, что к нему привело.

ЗАПРОС

```
{
  "firstName": "Пётр",
  "lastName":  "Сидоров",
  "birthDate": "2009-05-14",
  "region":    "77",
  "product":   "CASH_LOAN",
  "amount":    30000
}
```



В реальности ответ куда больше: сотня с лишним атрибутов — все сработавшие правила, все посчитанные модели с их значениями, применённые пороги, подобранные продукты. Полная структура — в материалах к заданию.

Именно с этими ответами вы и будете работать. Вызывать СПР не нужно — они уже собраны в датасете.

КРАТКОЕ ОПИСАНИЕ ТРЕБОВАНИЙ

Есть система, которая проверяет изменения в **системе принятия решений (СПР)** до выката на прод: одни и те же заявки прогоняются через две версии — старую и новую, — а потом ответы сравниваются, чтобы увидеть, что изменилось в решениях.

Система состоит из четырёх модулей. **Модули 1 и 2 уже работают. Вы делаете модули 3 и 4.**

МОДУЛЬ 1

Сборка и подготовка пулов ретро-заявок

Отбор исторических заявок и данных для повторного прогона

МОДУЛЬ 2

Исполнение тестов — прогон через СПР

Каждая заявка уходит в СПР дважды: версия с прода и версия с изменением

МОДУЛЬ 3

Приведение и сравнение данных

Разбор ответов, сопоставление срезов, витрина различий

МОДУЛЬ 4

Визуализация результатов в браузере

Сводная таблица, автопроверки, путь до заявки и атрибута

уже работает · результат выдан вам датасетом

ВАША ЗАДАЧА

МОДУЛЬ	ЧТО ПРОИСХОДИТ	КТО ДЕЛАЕТ
Модуль 1. Сборка и подготовка пулов ретро-заявок	Отбираются исторические заявки, собирается всё, что нужно для повторного прогона	уже работает
Модуль 2. Исполнение тестов — прогон через СПР	Каждая заявка отправляется в СПР дважды — в версию с прода и в версию с изменением. Ответы складываются в два среза	уже работает
Модуль 3. Приведение и сравнение данных	Ответы разбираются, приводятся к сравнимому виду, сопоставляются между срезами. Получается витрина различий	ваша задача
Модуль 4. Визуализация результатов в браузере	Сводная таблица, автопроверки, путь от числа до конкретной заявки и атрибута	ваша задача

Результат модулей 1 и 2 — это ваш вход: **датасет с двумя срезами ответов**. Ничего собирать и никуда ходить не нужно, данные выданы.

ЗАДАЧА

Что нужно построить

Один и тот же набор заявок уже прогнан через две версии стратегии: `BEFORE (MASTER)` — то, что работает на проде, и `AFTER (BETA)` — версия с изменением. Ответы выданы в датасете.

За 2 дня сделать многопользовательское приложение, которое превращает эти два среза в отчёт, по которому аналитик за минуту понимает, **что изменилось, у кого и из-за чего**.

В выданном датасете 2002 заявки — по ответу в каждом срезе. Прогоны на 20 000 заявок в этой работе обычное дело.

КОМУ НУЖЕН РЕЗУЛЬТАТ

Владельцу продукта — СПР-системы **Decision Flow**. Это он решает, когда правка стратегии уходит на прод, и ему нужно видеть последствия изменения **до выката**, а не после. Сегодня этот ответ собирают руками; ваше приложение должно давать его за минуты.

ОБЪЁМ РАБОТ

Что необходимо сделать

Реализовать многопользовательское приложение, выполняющее задачи процесса тестирования — **модуль 3 и модуль 4**.

МОДУЛЬ 3

Приведение и нормализация данных

- **Конфигурировать тесты:** выбрать датасет, полученный на предыдущих этапах, процедуры приведения и сравнения данных, набор автоматических проверок — сохранять конфигурацию и запускать повторно.

- Дать пользователям возможность **самостоятельно, на программном уровне, описывать разбор ответов** и преобразование их к базовому целевому формату. Базовая схема задана в материалах; она намеренно не универсальна, покрывает большинство ответов СПР с ограниченной глубиной вложенности и при необходимости может расширяться дополнительными полями пути или контекста как изменение общего контракта.
- Пользовательские процедуры приведения, сравнения, подготовки фактов для отчёта и автоматические проверки выполняются на **Python**. Для каждого скрипта доступны два способа подключения: написать или скорректировать код в интерактивной рабочей области, похожей на Jupyter Notebook, либо выбрать размещённый в системе файл `.py`, соответствующий утверждённому контракту входов и выходов. Для текущего задания версионирование пользовательского кода не требуется.
- Контракт скрипта определяет доступные входы, обязательный формат результата и диагностические артефакты. Например, процедура нормализации возвращает строки плоской таблицы, процедура агрегации — факты для отчёта, а проверка — строки PASS / FAIL.
- **Разрабатывать и применять наборы автоматических проверок**, написанных пользователями, и **объединять результаты всех проверок в отдельный отчёт** — единой таблицей PASS / FAIL. Рабочий набор проверок выдан: приложение должно уметь подключить и исполнить его, а также дать пользователю написать свои.
- После сопоставления значений система группирует данные по измерениям будущего отчёта и рассчитывает **counts** — количество заявок в каждой группе. Агрегированные факты и связанные с ними идентификаторы заявок передаются в сводный отчёт для агрегации и перехода к деталям.
- **Логировать выполнение всех алгоритмических этапов** и показывать статус выполнения запуска процедуры приведения данных. Из журнала запуска пользователь может провалиться к коду этапа, который исполнялся.

МОДУЛЬ 4

Визуализация результатов в браузере

- **По завершении формирования данных пользователь переходит к отчёту в браузере по ссылке.**
- **Отчёт по результатам прогона:** в нём отображаются все пользовательские проверки. Документ доступен **по ссылке** и **скачивается в HTML**. Из скачанного HTML можно провалиться в сводный отчёт приложения.
- **Сводный отчёт позволяет обозревать основные изменения в структурах данных и смещения значений к новым.** Таблицы с изменяемой раскладкой — показывают отличающиеся данные и их количества.
- **В сводном отчёте есть количественные показатели изменений значений конкретных атрибутов.** От числа изменившихся значений атрибутов пользователь проваливается до конкретной заявки и конкретного атрибута.
- Отчёт открывается, когда обработка **полностью завершена**. Показывать частичные результаты не нужно.

ЧТО ЗНАЧИТ «ОТЧЁТ» И «СВОДНАЯ ТАБЛИЦА»

Готовых макетов мы не даём — как показать различия, решаете вы. Но есть проверка на полезность: аналитик открывает результат прогона и **за минуту** отвечает на вопросы —

? сколько заявок изменилось и у скольких сменилось решение;

? какие атрибуты меняются чаще всего и кого это затронуло — продукты, партнёров, сегменты;

? из-за чего сменилось решение: новый стоп-фактор, другой балл, сдвинутый порог;

? где различия есть, а решение не изменилось — изменение прошло мимо, это тоже надо видеть.

Отчёт — это одна таблица, в которую стекаются строки всех пользовательских проверок, сколько бы их ни было и кто бы их ни написал. Каждая проверка возвращает свои строки, приложение объединяет их и выносит вердикт по каждой:

ПРОВЕРКА	СРЕЗ	АРГУМЕНТ	ЧИСЛА	МЕТРИКА	РЕЗУЛЬТАТ	ЗАЯВКИ
Отсутствующие атрибуты	AFTER (BETA)	—	0 / 500	—	PASS	—
Ошибки исполнения	BEFORE (MASTER)	—	4 / 500	0.8%	PASS	4 шт.
Ошибки исполнения	AFTER (BETA)	—	3 / 500	0.6%	PASS	3 шт.
Approval Rate продукта <10%	BEFORE (MASTER)	CASH_LOAN	214 / 500	32.1%	PASS	—
Approval Rate продукта <10%	AFTER (BETA)	CASH_LOAN	214 / 500	21.4%	FAIL	47 шт.
Продукт оффера нет на проде	AFTER (BETA)	CL_T60_22.9_PROMO	22 / 500	4.4%	FAIL	22 шт.
Версия скоркарты изменилась	AFTER (BETA)	TAKEUP_RATE_IP	495 / 500	v3 → v4	FAIL	495 шт.
Стоп-фактор пропал	AFTER (BETA)	AGELOW@21	41 → 0	-100%	FAIL	41 шт.
Стоп-фактор новый	AFTER (BETA)	DOCS_EXPIRED	63 / 500	12.6%	FAIL	63 шт.
Атрибут появился	AFTER (BETA)	decisionResult.businessDecision	500 / 500	—	FAIL	все
Порог сдвинулся	AFTER (BETA)	cutOff PD_RETAIL	128 / 500	0.55 → 0.58	FAIL	128 шт.
Различия есть, решение то же	AFTER (BETA)	—	117 / 500	23.4%	PASS	117 шт.
Ошибка справочника	AFTER (BETA)	MFO_ZAIMER_2026	2 / 500	0.4%	FAIL	2 шт.

Числа в колонке «Заявки» кликабельные — оттуда пользователь попадает к этим заявкам и дальше к конкретному атрибуту. Последняя строка бледная намеренно: срабатываний мало, уверенности в проблеме нет — такие строки нельзя подсвечивать так же, как настоящую поломку. Пример собран на другом прогоне, к вашему датасету эти числа отношения не имеют. Структура строки описана в схеме витрины.

Сводная таблица — это не таблица с итогами, а инструмент: раскладку в ней меняет пользователь, а не разработчик. Проще всего посмотреть на живом примере:

XLSX **example/Сводная_таблица_пример.xlsx** — витрина сравнения по 100 заявкам из вашего датасета и сводная таблица поверх неё. Откройте, встаньте на сводную и перетащите поля между строками, колонками и фильтром: данные те же, разрез другой. Кода приведения в файле нет — только его результат.

Excel здесь — **только способ показать идею**, а не образец реализации и не формат сдачи. Точно такую же сводную мы не требуем: **вы вправе предложить свой способ**. Важно другое — агрегировать сопоставленные данные по измерениям отчёта в факты **counts**: количество заявок в сочетании пути атрибута, значения или бакета, признака различия, среза и присоединённых полей входной заявки. Именно эти агрегированные факты питают сводный отчёт, а идентификаторы заявок в группировках обеспечивают переход до заявки и атрибута.

Техническая витрина строится как **вертикальный datamart**: нормализованные строки **BEFORE (MASTER)** и **AFTER (BETA)** объединяются вертикально, а принадлежность к срезу хранится в поле `SOURCE`. Сопоставление выполняется позднее по идентификатору заявки и пути атрибута. Горизонтальные колонки BEFORE / AFTER допустимы только как поясняющее представление в интерфейсе конкретной заявки или результата сравнения; они не являются форматом технического UNION и хранения datamart.

ТРЕБОВАНИЯ К РЕШЕНИЮ

Многопользовательность. Несколько пользователей работают одновременно, у каждого свои конфигурации и запуски.

Приведение данных может идти минутами — это нормально. Но пока оно идёт, интерфейс должен отвечать, а пользователь — видеть, на каком этапе расчёт и что происходит.

Отчёт должен быть быстрым. Он живёт поверх сотен тысяч строк, и здесь задержка бьёт по пользователю каждый раз: открытие, смена раскладки, фильтр, переход к заявке.

Стек и архитектура — ваш выбор. Ограничений нет. В том числе вы сами решаете, где именно выполняется расчёт и остаётся ли он в одном процессе на одной машине. Этот выбор вы будете обосновывать.

Обработка начинается до конца прогона. Ответы поступают постепенно, и приведение данных не обязано ждать последнего из них. НЕ ОБЯЗАТЕЛЬНО

АЛЬТЕРНАТИВНОЕ МНЕНИЕ ОТ ИИ

Прогон завершился: данные приведены, пользовательские проверки отработали. Дальше система обращается к языковой модели и получает независимый взгляд на результат — что в этих различиях выглядит подозрительно и на что стоит посмотреть аналитику. Это не замена проверкам, а второе мнение рядом с ними, в том же отчёте.

Витрину целиком в модель не отправить, поэтому придётся решить, что именно ей показать: агрегаты, выборку различий, результаты проверок. Это решение — часть задачи.

ВВОДНЫЕ

Что вам дано

ДАННЫЕ

- data/RUN-20260827-0001/ — **2002 заявки**, по ответу в каждом срезе. 4004 файла JSON, около 20 МБ
- Имя файла — идентификатор заявки. Один и тот же идентификатор есть в обоих срезах: это ключ сравнения
- В одном ответе **131–178 атрибутов**. Витрина сравнения выйдет порядка **290 тысяч строк**

МАТЕРИАЛЫ

- **docs/ФОРМАТ_ОТВЕТА.md** — что означают блоки ответа, какие поля технические, как устроены девять массивов и три режима их сравнения. **Читать до того, как писать разбор**
- **docs/СХЕМА_ВИТРИНЫ.md** — базовый целевой формат, к которому вы приводите данные, и схема результата проверки. Это контракт, по которому работают выданные проверки; набор структурных колонок рассчитан на большинство ответов СПР с ограниченной глубиной вложенности и при появлении нового типа ответа может быть расширен на уровне общего контракта
- **checks/** — пять рабочих автоматических проверок и контракт: что проверка получает и что возвращает
- **example/** — учебный пример приведения данных по этапам: два ответа, две плоские таблицы, вертикальный UNION, наглядное сравнение
- **examples/** — расширенный учебный набор: пошаговые данные модификации, агрегированные факты `counts`, Excel-демо и браузерный прототип сводного отчёта. Начните с README.md: это ориентир для понимания формата данных и результата, а не готовое решение
- **example/Сводная_таблица_пример.xlsx** — витрина по 100 заявкам и сводная таблица поверх неё: пример изменяемой раскладки
- **samples/** — несколько ответов из датасета, чтобы посмотреть глазами
- **docs/ГЛОССАРИЙ.md** — термины предметной области: стоп-фактор, скоркарта, порог, оффер

ЧЕГО НЕ ДАНО

- **Готового приложения или реализации разбора ответов** — ни кода, ни готовых процедур приведения и сравнения. Это ваша работа
- Скелета приложения, схемы хранения, рекомендаций по стеку

```
ЗАДАНИЕ/
├ Задание — Система анализа изменений.html эта карточка
│
├ data/
│  └ RUN-20260827-0001/
│     ├ manifest.json                    параметры прогона
│     └ RESULTS/
│        ├ BEFORE_MASTER/<APP_ID>.json    2002 ответа, версия с прода
│        └ AFTER_BETA/<APP_ID>.json       2002 ответа, версия с изменением
│
├ docs/
│  ├ ФОРМАТ_ОТВЕТА.md                    структура ответа и ловушки в данных
│  ├ СХЕМА_ВИТРИНЫ.md                    целевой формат — контракт
│  └ ГЛОССАРИЙ.md                        термины предметной области
│
├ checks/
│  ├ README.md                           контракт проверки и доступ к витрине
│  └ check_*.py                          пять рабочих проверок
│
├ example/
│  ├ README.md                           что происходит на каждом этапе
│  ├ 01_response_BEFORE.json             вход: ответ до изменения
```

```
│   ├─ 02_response_AFTER.json              вход: ответ после изменения
│   ├─ 03_normalized_BEFORE.csv            этап 1: плоская таблица BEFORE (MASTER), 12 строк
│   ├─ 04_normalized_AFTER.csv             этап 1: то же по AFTER (BETA), 14 строк
│   ├─ 05_union.csv                        этап 2: вертикальный UNION строк BEFORE (MASTER) и AFTER (BETA); SOURCE сохраняет срез
│   ├─ 06_comparison.csv                   этап 3: сравнение с признаком различия
│   └─ Сводная_таблица_пример.xlsx         витрина 100 заявок + сводная таблица
│
├─ examples/                               расширенный учебный набор модификации и отчёта
│   ├─ README.md                           этапы и ожидаемый результат
│   ├─ 01_response_BEFORE_MASTER_app1001.json
│   ├─ 02_response_AFTER_BETA_app1001.json
│   ├─ 03_normalized_BEFORE_app1001.csv
│   ├─ 04_normalized_AFTER_app1001.csv
│   ├─ 05_normalized_UNION_app1001.csv     вертикальный UNION; SOURCE хранит срез
│   ├─ 07_pivot_facts_demo.csv             facts counts для сводной таблицы
│   ├─ 09_pivot_report_demo.xlsx           Excel-демо сводной таблицы
│   └─ Прототип отчёта WebDataRocks.html   браузерный прототип отчёта
│
└─ samples/                                ответы из датасета для чтения глазами
```

ПРО УЧЕБНЫЙ ПРИМЕР

Пример приведения данных намеренно упрощён: в нём 12 строк на заявку, которые читаются глазами, — а не 139. Он показывает **метод**: как из документа получается плоская таблица и как две таблицы превращаются в сравнение.

Структура в примере другая, чем в датасете. Имена полей не совпадут — не тратьте время на их поиск. Формат ваших данных описан отдельно.

Базовые колонки нормализованной таблицы не являются универсальным описанием любого JSON/XML. Они покрывают целевой класс ответов СПР с ограниченной глубиной вложенности. Если для нового типа ответа базовой схемы недостаточно, общий контракт расширяется дополнительными полями, а не подгоняется под каждую произвольную структуру.

ПРО УЧЕБНЫЙ ПРОТОТИП ОТЧЁТА

Папка `examples/` содержит учебные результаты разбора и браузерный прототип сводного отчёта. Они показывают ожидаемую логику данных и один из способов их просмотра, но не задают технологию, интерфейс или архитектуру решения.

Не требуется повторять WebDataRocks или Excel-демо: команда выбирает свой способ визуализации.

И ЕЩЁ: У НАС ЕСТЬ ЗАЯВКИ, КОТОРЫХ ВЫ НЕ ВИДЕЛИ

Выданные 2002 заявки — это часть прогона. Остальные отложены, и на защите ваше приложение прогонят на них. Числа должны совпасть с нашим эталоном.

Смысл простой: подогнать результат под конкретные данные не получится, а корректное решение пройдёт проверку не заметив её.

ПРИЁМКА

Как мы будем проверять

Работа принимается, если приложение проходит весь путь целиком: датасет на входе — отчёт на выходе. Ниже то, что мы проверим на защите.

- **Полный прогон.** Приложение обрабатывает весь выданный датасет — 2002 заявки в двух срезах — и доводит его до отчёта.
- **Число заявок с различиями совпадает с эталоном.** Одно число, и оно сразу показывает, верно ли разобраны данные.
- **Различаются четыре типа изменения:** значение изменилось, атрибут появился, атрибут исчез, всё совпало.
- **Процедуры и проверки подключаются как конфигурация.** Пользователь выбирает интерактивный код рабочей области либо размещённый в системе файл `.py`, соответствующий утверждённому контракту, задаёт параметры и может заменить выбранный обработчик без изменения кода приложения.

- [x] Пользователь может написать или изменить процедуру и проверку в интерактивной рабочей области приложения и сразу применить её. Также он может выбрать размещённый в системе файл `.py`, реализующий утверждённый контракт входов и выходов.
- [x] **Выданные проверки подключаются и исполняются**, а их результаты собираются в один отчёт: он открывается по ссылке и скачивается в HTML.
- [x] **Сводная таблица с изменяемой раскладкой**, работающая поверх агрегированных фактов counts, и переходом от показателя к конкретной заявке и конкретному атрибуту.
- [x] **Журнал этапов** с переходом к коду этапа, который исполнялся, — в текущей его версии.
- [x] **Проект поднимается по README** на чистой машине. Мы запустим по вашей инструкции.
- [x] **Долгий этап не блокирует работу:** интерфейс отвечает, прогресс виден, понятно, что происходит.

Отдельно, уже как плюс: запуск переживает перезагрузку страницы — пользователь вернулся и видит состояние, а не пустой экран.

ЧТО ЗАМЕРЯЕМ	МИНИМУМ	ХОРОШО	ОТЛИЧНО
Приведение и сравнение всего датасета	≤ 5 мин	≤ 2 мин	**≤ 30 с**
Открытие сводной таблицы	≤ 5 с	≤ 2 с	**≤ 0,5 с**
Смена раскладки, фильтр, переход к заявке	≤ 5 с	≤ 1 с	**≤ 0,3 с**
Повторное открытие готового отчёта	≤ 5 с	≤ 1 с	**мгновенно**
Прогресс долгого этапа и неблокирующий запуск	обязательно	—	—

Демонстрируете на своём железе, поэтому абсолютные секунды между командами мы не сравниваем. Нас интересует другое: что вы замеряли, где оказалось узкое место, что дало ускорение и на какой машине это получено. Внятное объяснение здесь весит больше, чем красивая цифра.

ЧТО НЕ БУДЕТ ПРИНЯТО

- × **Скрипт вместо приложения.** Работающий разбор в консоли и CSV на диске — это половина задачи без второй половины.
- × **Разбор, зашитый под этот датасет.** Имена полей и пути в коде приложения, а не в процедуре пользователя: на следующем прогоне такое решение придётся переписывать.

СДАЧА

Как сдаём

Демо — **20 минут на команду** плюс вопросы. Сценарий один и тот же для всех: так решения можно сравнить между собой, а не только послушать про каждое.

Кроме демо сдаёте ссылку на репозиторий с README, по которому проект поднимается на чистой машине.

1. **Покажите конфигурацию прогона:** как пользователь настраивает прогон — выбирает датасет, процедуры приведения и сравнения, набор автоматических проверок. Для процедуры покажите выбор источника: интерактивная рабочая область или размещённый в системе файл `.py` по утверждённому контракту. Запустите.

   3 мин

2. **Пока идёт приведение — покажите журнал этапов:** что выполняется сейчас, что уже готово. Провалитесь из журнала в код этапа: рабочую область или выбранный файл `.py`, который исполняется.

   3 мин

3. **Дождитесь завершения и назовите числа:** сколько заявок с различиями и у скольких сменилось решение.

   1 мин

4. **Откройте сводную таблицу** — раскладка по атрибутам и типам изменения.

   3 мин

5. **Провалитесь от числа до данных:** список заявок, обзор конкретной заявки, конкретный атрибут — сравнение до и после.

   3 мин

6. **Откройте отчёт проверок:** все проверки одной таблицей с вердиктами, переход из строки к заявкам.

   2 мин

7. **Напишите новую проверку прямо в приложении**, примените её и покажите, что она попала в тот же отчёт.

   2 мин

8. **Скачайте документ-отчёт в HTML**, откройте скачанный файл и провалитесь из него в сводный отчёт приложения.

   1 мин

9. **Расскажите про стек и архитектуру:** почему выбрали именно это, какие были альтернативы, что замеряли, где оказалось узкое место и что дало ускорение.

   2 мин

ПРИНЦИПЫ ВЫПОЛНЕНИЯ

ИИ-ассистентами пользоваться можно — без ограничений и без оговорок.

Любые библиотеки, фреймворки и готовые pivot-компоненты можно. Изобретать сводную таблицу с нуля никто не просит.

Но выбор придётся объяснить. Решения, стек и архитектуру вы защищаете сами: почему так, какие были альтернативы и чем вы за это платите. Взяли готовый компонент — расскажите, почему именно его и что он не умеет.

Вопросы заказчику можно задавать всё время. Мы на связи в **МАКСе** в течение всего хакатона — спрашивайте про предметку, про данные, про ожидания от результата.